

Group Assignment: Machine Learning Competition

BEN BELHASSEN Mohamed Ali, CIAMPANA Lorenzo, FILESI Gianluca, NUBE Giacomo

1 Introduction

This report presents the process and outcomes of developing a predictive model for estimating adult weight, as part of a group assignment for the Machine Learning Competition. The project involved leveraging a dataset of 617 variables, encompassing both demographic and health-related attributes, to accurately predict the target variable, `WEIGHTLBTCA` (weight in pounds). The approach adopted for this task involved several key stages. Initially, exploratory data analysis (EDA) was conducted to understand the data distribution, identify missing values, and uncover patterns that could inform feature selection. Various machine learning algorithms, including regression-based models and tree-based techniques, were evaluated systematically to identify the best-performing model. Hyperparameter tuning and feature selection techniques were extensively utilized to enhance model performance. The final model was selected based on the minimization of mean squared error (MSE).

This report details the methodology, algorithms tested, feature engineering processes, and hyperparameter optimization strategies employed. Furthermore, we discuss the challenges encountered during the analysis, the rationale behind model selection, and insights derived from the results. By focusing on rigorous experimentation and adherence to machine learning best practices, this project not only addresses the prediction task but also highlights key considerations in applying machine learning to real-world datasets with high-dimensional features.

2 Data Cleaning and Processing

2.1 Feature Selection

The dataset contains 617 variables, making it crucial to carefully select the variables relevant to our analysis while removing those that do not contribute meaningfully or can not be used in a model. For instance, the variable `HHX` (Household X) is filled entirely with different string values (labels), rendering it non-influential. Consequently, we excluded this variable from our analysis.

Another type of not meaningful variable is a feature with a constant value. To systematically identify and remove such variables, we implemented an automated approach using a for loop. This loop iterated through all columns of the dataset, checking if the number of unique values was 1.

```
1 for each in df.columns:
2     if df[each].nunique() == 1:
3         df.drop(each, axis=1, inplace=True)
```

If this condition was met, it indicated that the variable contained only constant values, and it was subsequently removed. This process ensured the dataset was streamlined and focused solely on variables that could potentially influence the analysis.

At the end of this phase, our dataset contains 608 features.

2.2 NaN Values: Removal, Replacement

To ensure the dataset is clean and meaningful, we addressed columns containing a significant number of missing values (NaNs). When the number of empty rows in a column significantly exceeds the number of non-empty rows, that feature likely holds limited predictive value. Consequently, we opted to remove such columns based on a predefined threshold.

Specifically, we set a threshold of 10% of the dataset length, meaning any column with more than 10% of its entries missing was dropped. This process was automated with a loop that iterated through all features in the dataset, checked the proportion of missing values, and removed those exceeding the threshold.

```
1 features = df.columns
2
3 for feature in features:
4     if df[feature].isna().sum() > 0.1 * len(df):
5         df.drop(feature, axis=1, inplace=True)
6 features = df.columns
```

To handle the remaining features with missing values, we used the K-Nearest Neighbors (KNN) Imputer to fill in the gaps. This method estimates missing values based on the values of the nearest neighbors in the dataset, ensuring the imputed values align with the overall structure of the data. Specifically, we set the number of neighbors (`n_neighbors`) to 3, allowing the imputer to consider the three closest points for filling in missing data.

After imputing, we verified the dataset to ensure no inconsistencies in the variables. To do this, we calculated the absolute correlation matrix (**Sigma**) for the features and checked its diagonal. If the sum of the diagonal values equals the number of variables, it confirms that the features exhibit no anomalies or issues in their correlations.

At the end of this phase, our dataset contains 207 features.

2.3 Multicollinearity

To address multicollinearity issues in the dataset, we employed a two-step process. First, we removed features with extremely high correlation coefficients to eliminate redundancy among variables. Specifically, we set the diagonal of the absolute correlation matrix (**Sigma**) to zero and iterated through each column to identify and drop features with a maximum correlation coefficient greater than 0.9. This ensured that highly correlated features were excluded from the dataset, retaining only those with more independent relationships.

```
1 np.fill_diagonal(Sigma.values, 0)
2 for each in Sigma.columns:
3     if Sigma[each].max() > 0.9:
4         df_imputed.drop(each, axis=1, inplace=True)
5 features = df_imputed.columns
```

Afterward, we conducted a deeper analysis of multicollinearity using the Variance Inflation Factor (VIF). We defined the target variable `WEIGHTLBTC_A` and separated it from the feature set. The feature matrix **X** was scaled using `StandardScaler` for consistency and normalization, and an intercept column was added for the VIF calculation.

```
1 target = ['WEIGHTLBTC_A']
2 features.drop(target)
3 X = df_imputed[features]
4 y = df_imputed[target].values.ravel()
5 scaler = StandardScaler()
6 X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
7 X_scaled['Intercept'] = 1
```

To compute VIF, we implemented a function that iterates over each feature in the scaled dataset, calculating the VIF for each. Variables with a VIF greater than 10, indicative of high multicollinearity, were iteratively removed, starting with the feature with the highest VIF. This process was repeated until all features had a VIF value below the threshold, ensuring a multicollinearity-free dataset.

```
1 def calculate_vif(X):
2     vif_data = pd.DataFrame()
3     vif_data["Variable"] = X.columns
4     vif_data["VIF"] = [variance_inflation_factor(X, i) for i in range(X.shape[1])]
5     return vif_data
```

Finally, we removed the intercept column from the scaled dataset, leaving a refined feature set for modeling. The resulting dataset had a shape of (20340, 207).

2.4 Dimension Reduction

To reduce the dimensionality of the dataset while retaining the majority of its variance, we decided to set a threshold of 80% for the cumulative sum of the explained variances. Using the correlation matrix (**Sigma**) of the scaled feature set (`X_scaled`), we computed its eigenvalues and eigenvectors. The eigenvalues were sorted in descending order to assess their contribution to the total variance.

We iteratively summed the eigenvalues, starting with the largest, until the cumulative explained variance exceeded 80%. This process determined the optimal number of components needed for Principal Component Analysis (PCA).

```
1 Sigma = X_scaled.corr()
2 L, V = np.linalg.eig(Sigma)
3 L = np.sort(L)[::-1]
4 i = 1
5 S = 0
6 while S < 0.8:
7     S = L[:i].sum() / L.sum()
8     i += 1
```

Using this calculated number of components, we applied PCA to the scaled dataset. The final transformed dataset (`data_pca`) retained a reduced dimensionality while preserving the variance threshold, resulting in a shape of (20340, 66).

```
1 pca = PCA(n_components=i-1)
2 data_pca = pca.fit_transform(X_scaled)
```

Since our aim is having the model with the smallest MSE, we will train and test the models on both the dataset before PCA and the dataset after PCA to evaluate which approach yields better performance.

3 Model selection

3.1 Cross validation

The `train_test_split` function was used to divide the dataset into training and testing subsets to ensure the model is evaluated on unseen data, providing a more accurate measure of its generalization performance. This method ensures a randomized and controlled split, which helps avoid bias and overfitting by assessing the model's ability to perform on data it has not encountered during training. By allocating a portion of the data for testing, we create a clear distinction between the training phase and the evaluation phase, which is critical for building robust and reliable models.

We used `train_test_split` instead of K-Fold Cross Validation or Leave-One-Out Cross Validation because it is computationally more efficient and sufficient for our current needs.

K-Fold and LOO are often used when we want to evaluate the model's performance across multiple splits or ensure robustness, but they require training the model multiple times, which can be computationally expensive, especially with large datasets or complex models.

In contrast, `train_test_split` provides a simpler and faster way to achieve a reliable estimate of model performance by dividing the data into a single training and testing set. This approach is particularly appropriate when the dataset is sufficiently large as in our case, as the single split is usually representative of the overall distribution.

We decided to split the dataset as 70% as training set and 30% as test set.

```
1 feature_train, feature_test, target_train, target_test = train_test_split(X_scaled, y, test_size=0.3)
```

3.2 Models comparison

To evaluate and compare the performance of different models, we have implemented a function that takes a model as its input and returns the Mean Squared Error (MSE) as its output. This function serves as a standardized tool for assessing how well each model performs in terms of its predictive accuracy.

```
1 def model_MSE(model):
2     model.fit(feature_train, target_train)
3     target_pred = model.predict(feature_test)
4
5     MSE = mean_squared_error(target_test, target_pred)
6     return MSE
```

Model	MSE
Decision Tree Classifier	0.003
Bagging Classifier	0.005
Random Forest Classifier	161.161
Ada Boosting Classifier	1051.531
Gradient Boosting Classifier	11.994
XGB Regressor	0.002
Linear Regression	3.149×10^{-26}
Lasso $\alpha = 1$	0.994
Ridge $\alpha = 1$	2.841×10^{-5}

Table 1: MSE for the models tested

3.3 Models comparison - PCA dataset

As we already discussed about, we have performed a PC analysis on our dataset. PCA allows to reduce the computational time and to better handle large datasets. However, it is based on an approximation. Since our aim is having the model with the smallest MSE, these approximations should cause an increase in MSE.

Model	MSE
Decision Tree Classifier	921.577
Bagging Classifier	896.662
XGB Regressor	150.911
Linear Regression	195.771
Lasso $\alpha = 1$	209.770
Ridge $\alpha = 1$	150.909

Table 2: MSE for the models tested on PCA dataset

As expected, all the models show a worse MSE. For this reason, we have decided to not use the PCA dataset.

4 Final consideration

Given the results in table 1, the best model is the Linear Regression. However, such small result can indicate an overfitting problem in our model. For this reason, it might be better to use as model the XGB Regressor with an MSE of 0.00178.